Entwicklungsdokumentation

Temperaturüberwachungssystem

Lernfeld 7

Inhaltsverzeichnis

Entwicklungsdokumentation 1
Temperaturüberwachungssystem 1
Inhaltsverzeichnis 2
Einleitung 6
1. MQTT – Protokollübersicht 6
Begriffe 6
MQTT-Architektur 7
2. Installation des MQTT-Brokers und der Infrastruktur 7
Ansatz 7
docker-compose.yml 8
Installationsschritte 9
Umgebungsvariablen (.env) 10
3. MQTT-Broker-Konfiguration und Client-Tests 10
Mosquitto-Konfiguration (mosquitto.conf) 10
MQTT-Client-Test mit MQTT Explorer 10
4. ESP32 als MQTT-Client mit DS18B20 11
Hardwareaufbau 11
Funktionsweise 11

Quellcode (main.cpp snippet)

```
void setup() {
  Serial.begin(115200);
  delay(1000);

  Serial.println("\n\nESP32 Engine starting...");

  initDisplay();
  initLEDs();
  sensors.begin();
  initWiFi();
  initServer();

  measureTemperature();
  lastMeasure = millis();
}

void loop() {
  server.handleClient();

  if (millis() - lastMeasure >= MEASURE_INTERVAL) {
    measureTemperature();
    lastMeasure = millis();
  }

  delay(50);
}
```


..... 12

5. JSON – Kurzübersicht 12

Was ist JSON? 12

Syntax-Regeln 12

Beispiel – MQTT-Payload unseres ESP32 12

Verwendung im Projekt 13

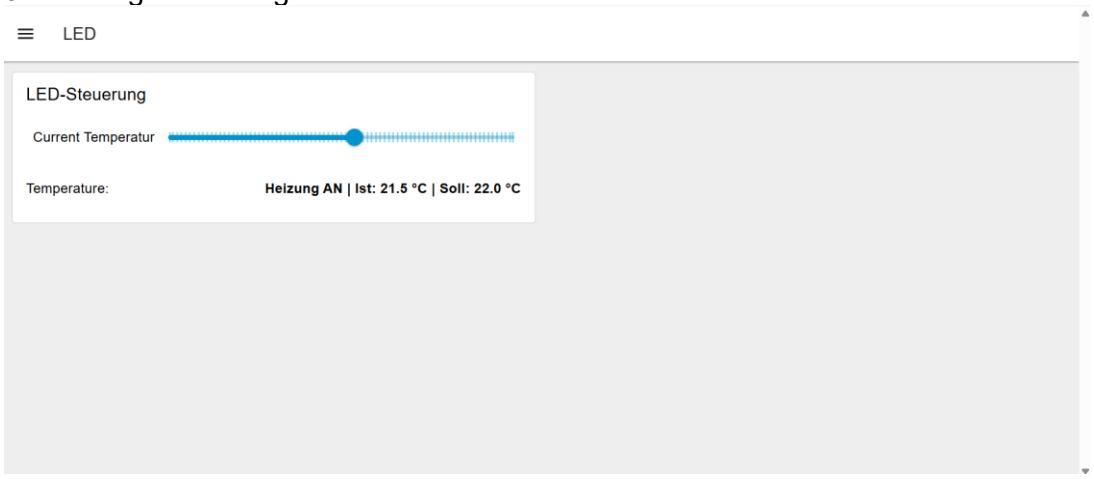
6. Node-RED – Installation und erste Tests 13

Installation 13

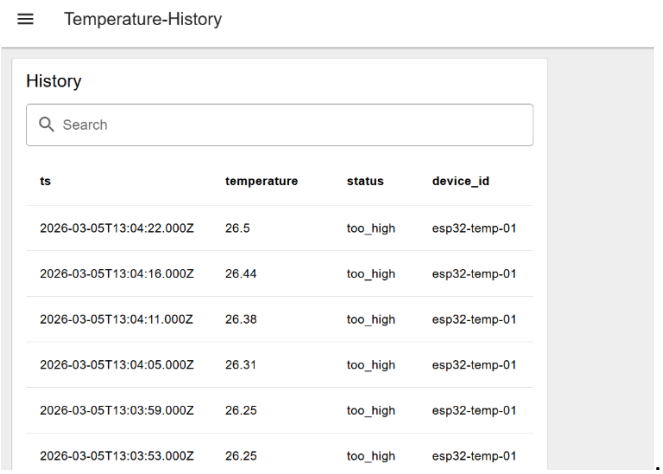
Zusätzlich installierte Nodes 13

8. Datenausgabe als Chart und Text 15

9. Heizungssteuerung



..... 16
10. Datenbankabfrage mit Filtern über das Dashboard 16
MySQL-Node Einstellungen 16
11. Anzeige der Temperaturdaten 17



..... 17

```
SELECT ts, temperature, status, device_id
FROM temperature_readings
WHERE device_id     = ?
  AND ts           >= ?
  AND ts           <= ?
  AND temperature >= ?
  AND temperature <= ?
ORDER BY ts DESC
LIMIT ?;
```

..... 17
Fazit 17
Erreichter Projektstatus 17
Lernergebnisse 18
Mögliche Erweiterungen 18
Zusammenfassung 18

Einleitung

Dieses Projekt befasst sich mit der Entwicklung eines modernen Temperaturüberwachungssystems, das verschiedene Technologien der internetverbundenen Geräte (IoT) miteinander verbindet. Das System integriert Hardwarekomponenten, standardisierte Kommunikationsprotokolle und eine webbasierte Benutzeroberfläche zu einer umfassenden Lösung für dezentrale Temperaturmessung und -verwaltung.

Das Projekt wird im Kontext des Lernfeldes 7 durchgeführt und verfolgt das Ziel, praktische Erfahrungen mit modernen IoT-Technologien zu sammeln. Dabei werden die einzelnen Komponenten einer Temperaturüberwachung, die in vorherigen Lerneinheiten behandelt wurden, zu einem funktionsfähigen Gesamtsystem zusammengefügt.

Die Dokumentation hält alle wesentlichen Schritte fest, angefangen von der theoretischen Grundlage bis zur praktischen Implementierung und Inbetriebnahme des Systems. Sie dient sowohl als Nachweis der durchgeführten Arbeiten als auch als Nachschlagewerk für zukünftige Wartungs- und Erweiterungsarbeiten.

1. MQTT – Protokollübersicht

Begriffe

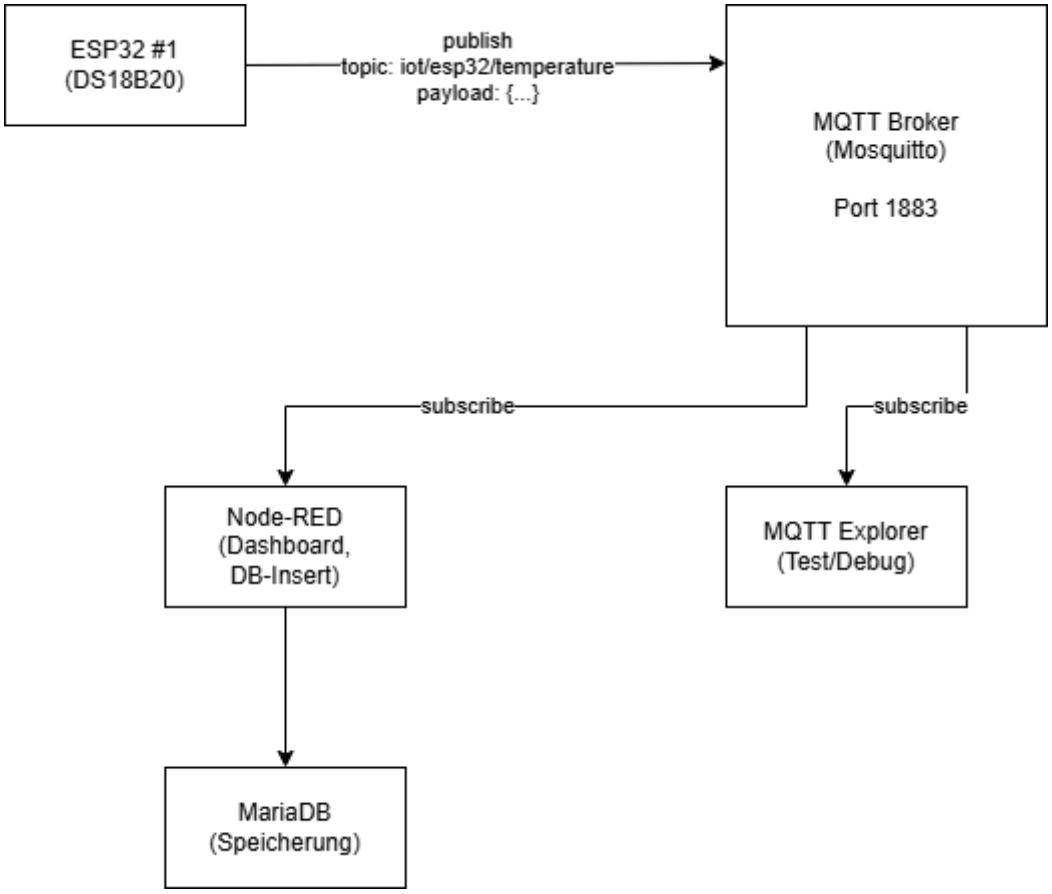
Begriff	Beschreibung
MQTT	Message Queuing Telemetry Transport – leichtgewichtiges Publish/Subscribe-Protokoll für M2M-Kommunikation (Machine to Machine), entwickelt für Geräte mit begrenzten Ressourcen und instabile Netzwerke.
Broker / Server	Zentrale Vermittlungsstelle. Empfängt alle Nachrichten von Publishern und leitet sie an die passenden Subscriber weiter. In unserem Projekt: Eclipse Mosquitto.
Client	Jedes Gerät, das sich mit dem Broker verbindet, egal ob es sendet (publish) oder empfängt (subscribe). Beispiele: ESP32, Node-RED, MQTT Explorer.
Publish	Ein Client veröffentlicht eine Nachricht unter einem bestimmten Topic an den Broker.
Subscribe	Ein Client abonniert ein Topic beim Broker und erhält ab dann alle dort veröffentlichten Nachrichten.
Topic	Hierarchischer Adresspfad einer Nachricht, z. B. tms/esp32-temp-01/temperature. Funktioniert wie ein Ordnerpfad mit / als Trennzeichen.
Payload	Der eigentliche Nachrichteninhalt (Nutzlast), z. B. ein JSON-String mit Temperaturwert und Status.

QoS (Quality of Service)	Zustellgarantie-Stufe: 0 = höchstens einmal (fire & forget), 1 = mindestens einmal (mit ACK), 2 = genau einmal (4-Wege-Handshake). In unserem Projekt verwenden wir QoS 0, da Temperaturdaten regelmäßig gesendet werden und ein einzelner Verlust unkritisch ist.

MQTT-Architektur

Die MQTT-Architektur folgt einem Publish/Subscribe-Modell:

- ESP32-Sensoren publizieren Temperaturdaten unter Topics wie tms/esp32-temp-01/temperature
- Der MQTT-Broker (Eclipse Mosquitto) fungiert als zentrale Vermittlungsstelle
- Node-RED, MQTT Explorer und weitere Clients subscriben diese Topics und erhalten die Nachrichten
- Die Daten fließen in die MariaDB-Datenbank zur Speicherung und Auswertung



2. Installation des MQTT-Brokers und der Infrastruktur

Ansatz

Die gesamte Infrastruktur wurde plattformunabhängig mit Docker und Docker Compose realisiert. Dies ermöglicht einen identischen Betrieb sowohl auf einem Raspberry Pi als auch auf einem Windows-/Mac-Laptop.

Folgende Dienste werden als Container bereitgestellt:

Dienst	Image	Port(s)	Funktion
Mosquitto	eclipse-mosquitto:latest	1883, 9001	MQTT Broker
MariaDB	mariadb:latest	3307 → 3306	Datenbank für Messwerte
phpMyAdmin	phpmyadmin:latest	8081 → 80	Web-GUI für DB
Node-RED	nodered/node-red:latest	1880	Flow + Dashboard

docker-compose.yml

```yaml
services:
  mosquitto:
    image: eclipse-mosquitto:latest
    container_name: mqtt-broker
    ports:
      - "1883:1883"
      - "9001:9001"

    volumes:
      - ./mosquitto/config:/mosquitto/config
      - ./mosquitto/data:/mosquitto/data
      - ./mosquitto/log:/mosquitto/log

    networks:
      - iot-network

    restart: unless-stopped

  mariadb:
    image: mariadb:latest
    container_name: temperature-db
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
    ports:
      - "3307:3306"

    volumes:
      - mariadb-data:/var/lib/mysql

    networks:
      - iot-network

    restart: unless-stopped
    healthcheck:
      test: [ "CMD-SHELL", "mariadb-admin ping -h localhost -p$MYSQL_ROOT_PASSWORD || exit 1" ]
      interval: 10s
      timeout: 3s
      retries: 10
  phpmyadmin:
    image: phpmyadmin:latest
    container_name: phpmyadmin
    ports:
```

```yaml
    ports:
      - "8081:80"
    environment:
      PMA_HOST: mariadb
      PMA_PORT: 3306
      PMA_USER: ${MYSQL_USER}
      PMA_PASSWORD: ${MYSQL_PASSWORD}
    networks:
      - iot-network
    depends_on:
      mariadb:
        condition: service_healthy
    restart: unless-stopped


  node-red:
    image: nodered/node-red:latest
    container_name: node-red
    ports:
      - "1880:1880"

    environment:
      - TZ=${TIMEZONE}

    volumes:
      - node-red-data:/data

    networks:
      - iot-network

    restart: unless-stopped
    depends_on:
      - mosquitto
      - mariadb

networks:
  iot-network:
    driver: bridge

volumes:
  mariadb-data:
  node-red-data:
```

Installationsschritte

1. Docker und Docker Compose auf dem Zielgerät installieren (Raspberry Pi oder Laptop)
2. Projektverzeichnis mit docker-compose.yml, .env-Datei und Mosquitto-Konfiguration anlegen
3. Start aller Dienste: docker-compose -p tms up -d
4. Prüfung, ob alle Container laufen: docker ps

Umgebungsvariablen (.env)

Die sensiblen Zugangsdaten werden in einer .env-Datei ausgelagert (nicht ins Git-Repository eingecheckt):

```
# MariaDB Konfiguration
MYSQL_ROOT_PASSWORD=pass
MYSQL_DATABASE=temperaturedb
MYSQL_USER=admin
MYSQL_PASSWORD=pass

# Node-RED Konfiguration
TIMEZONE=Europe/Berlin

# MQTT-Broker
MQTT_BROKER=mosquitto
MQTT_PORT=1883
```

3. MQTT-Broker-Konfiguration und Client-Tests

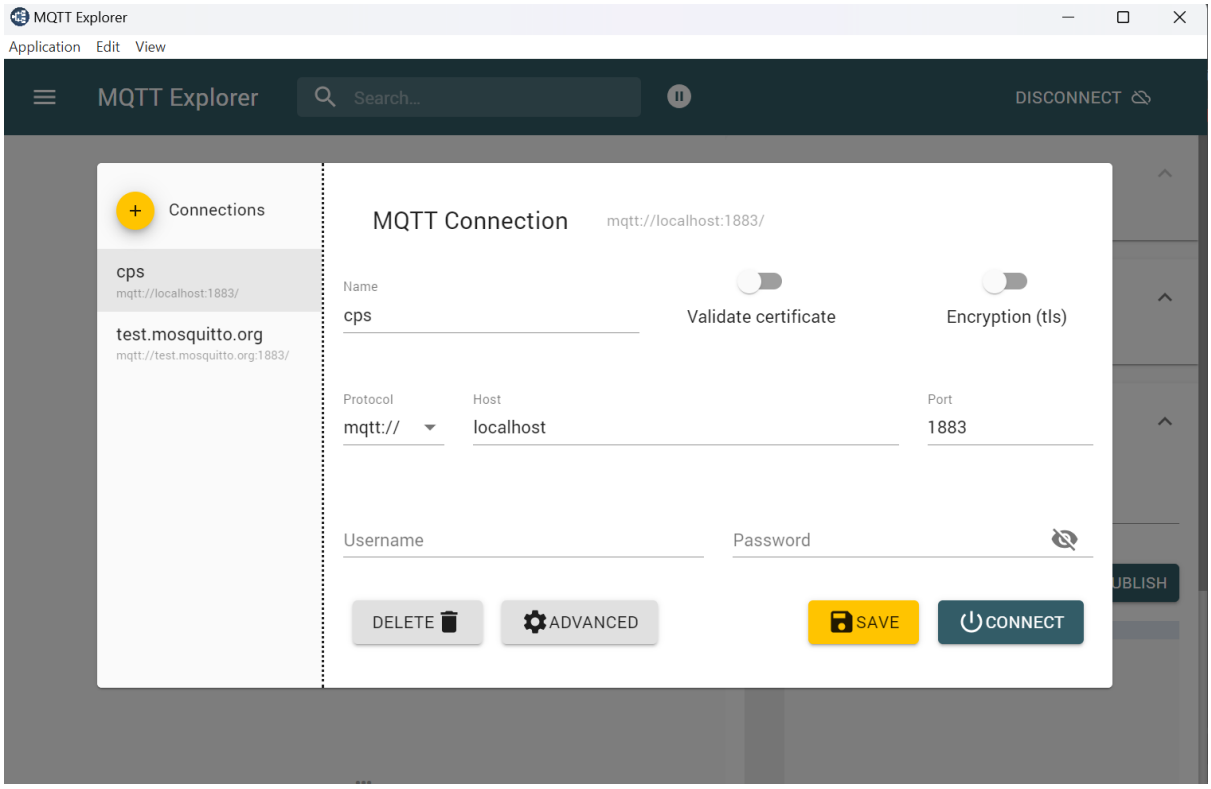
Mosquitto-Konfiguration (mosquitto.conf)

listener 1883 0.0.0.0 protocol mqtt listener 9001 0.0.0.0 protocol websockets allow_anonymous true persistence true persistence_location /mosquitto/data/

Einstellung	Bedeutung
listener 1883 0.0.0.0	MQTT-Port auf allen Netzwerk-Interfaces
listener 9001 0.0.0.0	WebSocket-Port (z. B. für browserbasierte Clients)
allow_anonymous true	Keine Authentifizierung erforderlich (Testbetrieb)
persistence true	Nachrichten werden bei Broker-Neustart beibehalten

MQTT-Client-Test mit MQTT Explorer

- Verbindung: Host = IP des Docker-Hosts, Port = 1883
- Publish-Test: Topic tms/test, Payload {"temperature": 22.5, "status": "optimal"}
- Subscribe-Test: Topic tms (Wildcard für alle Unter-Topics)



4. ESP32 als MQTT-Client mit DS18B20

Hardwareaufbau

- Mikrocontroller: ESP32
- Temperatursensor: DS18B20 (an GPIO 4, mit 4,7 kΩ Pull-up)
- LEDs: Grün (GPIO 14), Rot (GPIO 12), Gelb (GPIO 13)
- OLED-Display: SSD1306 128×64 (I^2C: SDA = GPIO 21, SCL = GPIO 22)

Funktionsweise

Funktion	Beschreibung
Temperaturmessung	Alle 5 Sekunden über DallasTemperature-Bibliothek
Status-LEDs	Grün = optimal (19–26 °C), Rot = zu hoch, Gelb = zu niedrig
OLED-Anzeige	Aktuelle Temperatur, Bereich, Status, WiFi-Info
REST-API	JSON-Endpunkte /api/temperature und /api/config
WiFi	Eigenes WLAN (WLAN-ESP) oder Verbindung zum lokalen Netzwerk

Quellcode (main.cpp snippet)

```
void setup() {
  Serial.begin(115200);
  delay(1000);

  Serial.println("\n\nESP32 Engine starting...");

  initDisplay();
  initLEDs();
  sensors.begin();
  initWiFi();
  initServer();

  measureTemperature();
  lastMeasure = millis();
}

void loop() {
  server.handleClient();

  if (millis() - lastMeasure >= MEASURE_INTERVAL) {
    measureTemperature();
    lastMeasure = millis();
  }

  delay(50);
}
```

5. JSON – Kurzübersicht

Was ist JSON?

JSON (JavaScript Object Notation) ist ein leichtgewichtiges, textbasiertes Datenformat zum Austausch strukturierter Daten. Es ist sprach- und plattformunabhängig und wird im IoT-Bereich als Standard für MQTT-Payloads und REST-APIs verwendet.

Syntax-Regeln

- Daten werden als Schlüssel-Wert-Paare geschrieben: "key": value
- Objekte stehen in geschweiften Klammern {}
- Arrays stehen in eckigen Klammern []
- Erlaubte Werttypen: String, Number, Boolean, null, Object, Array

Beispiel – MQTT-Payload unseres ESP32

{

"device_id": "esp32-temp-01",
"temperature": 23.5,
"status": "optimal",
"minTemp": 19.0,
"maxTemp": 26.0
}

Verwendung im Projekt

Stelle	Format	Richtung
ESP32 → MQTT Broker	JSON-Payload im MQTT-Topic	Publish
Node-RED ← MQTT Broker	JSON wird geparst (json-Node)	Subscribe
ESP32 REST-API	JSON-Response auf HTTP GET	Request/Response
Node-RED Flow Export	Gesamter Flow als JSON-Datei	Export/Import

6. Node-RED – Installation und erste Tests

Installation

Node-RED wird als Docker-Container über die docker-compose.yml (siehe Abschnitt 2) gestartet.

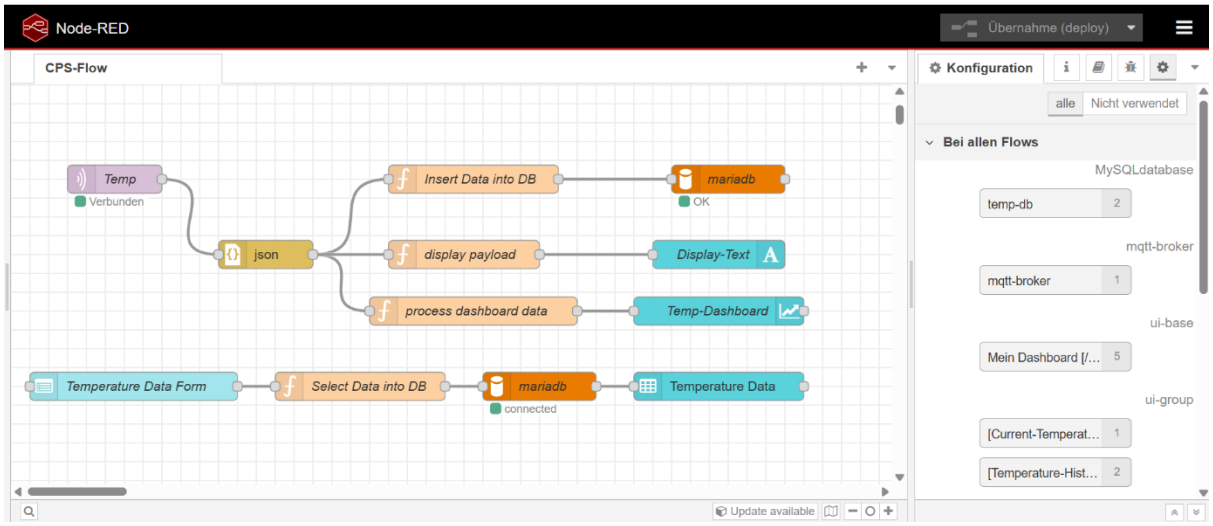
docker-compose -p tms up -d

Erreichbar unter: http://<host-ip>:1880

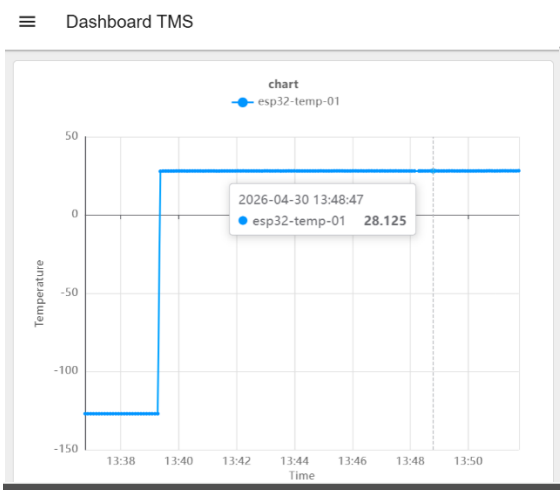
Zusätzlich installierte Nodes

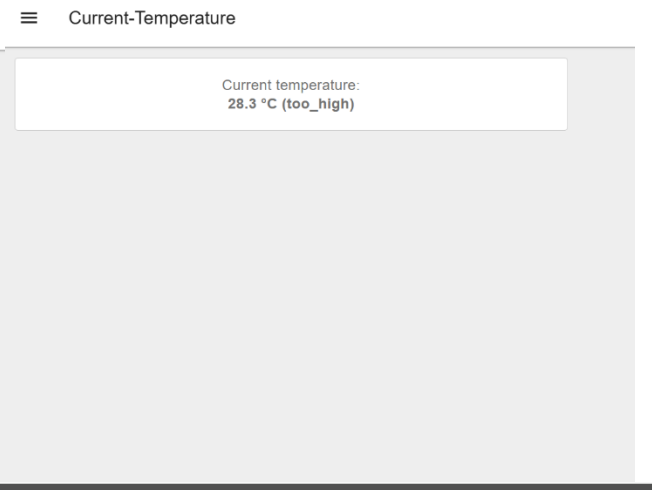
In der Node-RED-Oberfläche über Manage Palette installiert:

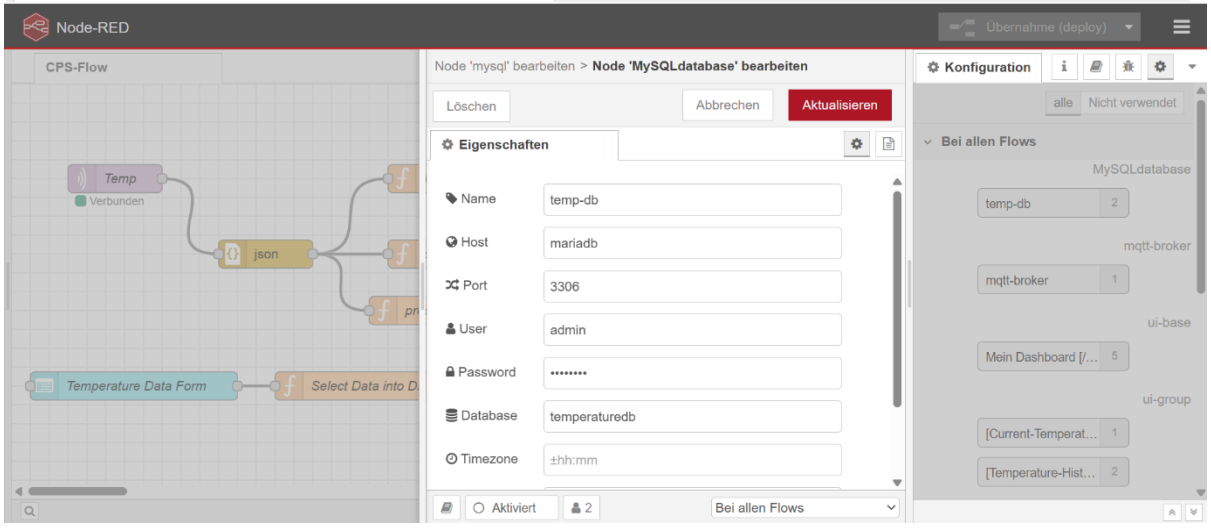
- node-red-node-mysql – MySQL/MariaDB-Anbindung
- node-red-dhboard – Dashboard-UI-Elemente (Gauge, Chart, Text etc.)


Node-RED
Übernahme (deploy)
CPS-Flow
Konfiguration
alle
Nicht verwendet
Bei allen Flows
MySQLdatabase
temp-db
2
mqtt-broker
mqtt-broker
1
ui-base
Mein Dashboard [/...
5
ui-group
[Current-Temperat...
1
[Temperature-Hist...
2
Temp
Verbunden
Insert Data into DB
mariadb
OK
json
display payload
Display-Text
process dashboard data
Temp-Dashboard
Temperature Data Form
Select Data into DB
mariadb
connected
Temperature Data
Update available

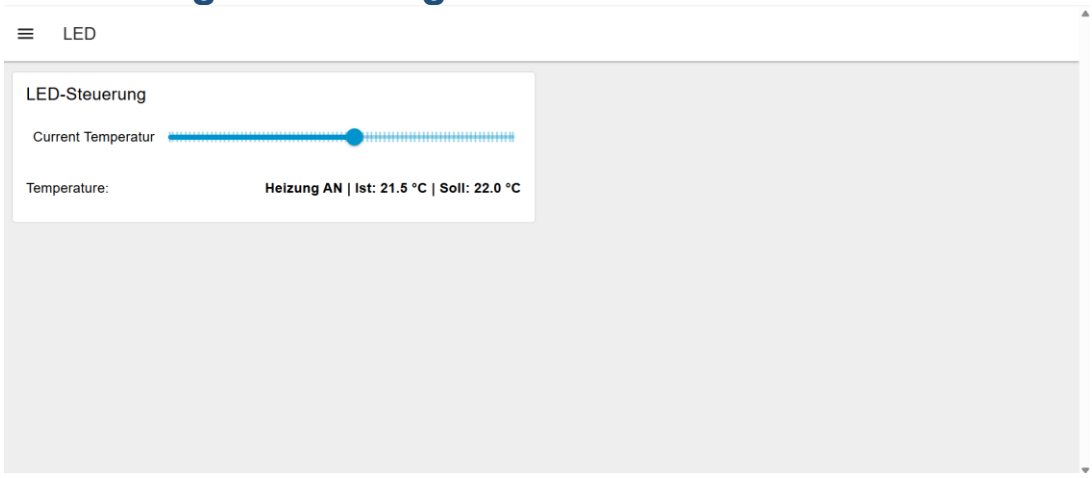
8. Datenausgabe als Chart und Text







9. Heizungssteuerung

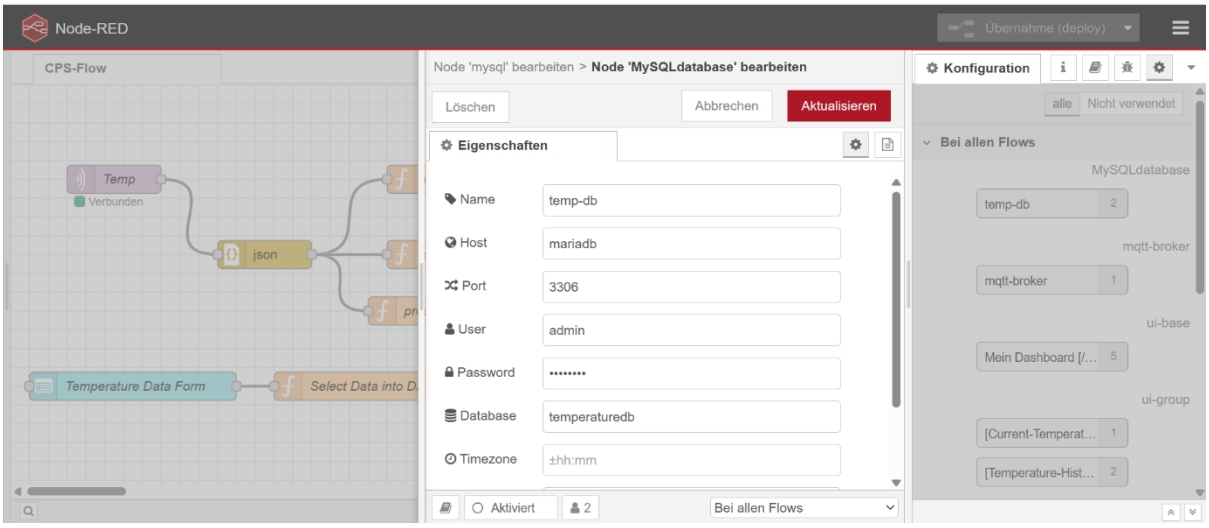


Über das Node-RED Dashboard kann eine simulierte Heizung (LED am ESP32) per MQTT gesteuert werden.

10. Datenbankabfrage mit Filtern über das Dashboard

MySQL-Node Einstellungen

Einstellung	Wert
Host	mariadb (Docker-Container-Name)
Port	3306
Database	temperature_db
User / Password	aus .env-Datei



11. Anzeige der Temperaturdaten

Über Dashboard-Eingabefelder können folgende Filter gesetzt werden:

- Device-ID
- Status
- Zeitraum (Start / Ende)
- Temperaturbereich (Min / Max)
- Ergebnislimit

Temperature-History

History

Search

ts	temperature	status	device_id
2026-03-05T13:04:22.000Z	26.5	too_high	esp32-temp-01
2026-03-05T13:04:16.000Z	26.44	too_high	esp32-temp-01
2026-03-05T13:04:11.000Z	26.38	too_high	esp32-temp-01
2026-03-05T13:04:05.000Z	26.31	too_high	esp32-temp-01
2026-03-05T13:03:59.000Z	26.25	too_high	esp32-temp-01
2026-03-05T13:03:53.000Z	26.25	too_high	esp32-temp-01

```sql
SELECT ts, temperature, status, device_id
FROM temperature_readings
WHERE device_id     = ?
  AND ts           >= ?
  AND ts           <= ?
  AND temperature >= ?
  AND temperature <= ?
ORDER BY ts DESC
LIMIT ?;
```

Fazit

Das Temperaturüberwachungssystem (TMS) demonstriert erfolgreich die Integration moderner IoT-Technologien in ein funktionierendes Gesamtsystem. Alle Komponenten arbeiten zusammen, um eine zuverlässige und wartbare Lösung zu schaffen.

Erreichter Projektstatus

- ✓ Hardware-Setup vollständig: ESP32 mit DS18B20-Sensor, Status-LEDs und OLED-Display
- ✓ MQTT-Infrastruktur live: Mosquitto-Broker mit Docker-Container auf Raspberry Pi/Laptop

- ✓ Datenbank-Integration: MariaDB speichert alle Messwerte mit Zeitstempel und Status
- ✓ Dashboard und Visualisierung: Node-RED präsentiert Daten in Echtzeit mit Filtern
- ✓ Versionskontrolle: Git-Repository ermöglicht Zusammenarbeit und Nachverfolgung

Lernergebnisse

- Tiefes Verständnis des MQTT-Protokolls und seine Vorteile für IoT-Kommunikation
- Praktische Erfahrung mit Microcontroller-Programmierung (ESP32) und Sensoranbindung
- Kompetenz in Container-Technologien (Docker, Docker Compose) und deren Deployment
- Fähigkeit zur Datenbank-Integration und SQL-Abfragen mit dynamischen Filtern
- Praktische Node-RED-Erfahrung für Flow-basierte Datenverarbeitung und Dashboards

Mögliche Erweiterungen

- Authentifizierung und Verschlüsselung für den MQTT-Broker (TLS/SSL)
- Mehrere ESP32-Sensoren mit individueller Identifikation und Verwaltung
- Alarm-System: Benachrichtigungen per E-Mail oder Telegram bei Temperaturabweichungen
- Hochfahren des Systems auf Cloud-Plattformen (AWS, Azure, Google Cloud)
- Integration weiterer Sensoren (Luftfeuchtigkeit, Luftdruck, etc.)

Zusammenfassung

Dieses Projekt hat gezeigt, dass die Kombination aus klassischen IoT-Konzepten (Hardware, Protokolle, Datenbanken) und modernen Werkzeugen (Docker, Node-RED, MQTT) ein robustes und wartbares System schaffen kann. Die Dokumentation dient zukünftigen Entwicklern als verlässliche Referenz und das System als Basis für weitere IoT-Experimente im Lernkontext.

Die Implementierung zeigt Best Practices in Bereichen wie Code-Struktur (APIs mit Interfaces), Konfigurationsmanagement (Umgebungsvariablen) und sichere Datenbankverwaltung. Mit diesem Grundgerüst lassen sich beliebig komplexere Temperatur- und andere Sensorüberwachungssysteme aufbauen.